

Lab0 GitLab 实验报告

姓名：何思源

仓库地址：<https://github.com/YuanShiWuYu123/GitLab>

一、文档问题回答

1. 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

有多人协同开发的经历。最初是通过传递压缩包的形式来进行分工协作，一个人改好之后压缩包发给下一个人，下一个接着改代码，改好之后再打包。实际上是无法同时写代码的，比较麻烦。后来接触GitHub之后，开始使用分支管理来进行合作，方便很多。

2. Git 为什么要设计“暂存-提交”两个步骤？

如果只有提交一个步骤，那么一次提交就把工作区里所有的改动都记录下来了。但是实际开发中通常会同时改很多地方，比如一边修 bug 一边加新功能。如果把修bug和新功能的修改混在一次提交里，历史记录就会混乱，回退时也没法只撤销其中一部分。有了暂存区之后，就可以自己挑选这次要提交哪些内容，用 `git add` 把相关的改动放进暂存区，提交时只记录这部分，剩下的留在工作区里继续改，下一次再单独提交。这样每次提交都只做一件事，历史清晰，也方便定位问题。

3. `git branch` 和 `git branch -a` 的区别是什么？

`git branch` 只列出本地分支，`git branch -a` 除了本地分支，还会列出远程跟踪分支。

在本次实验的仓库里，两条命令的输出如下：

```
Admin@MacBook-Pro GitLab % git branch
feature
* main
Admin@MacBook-Pro GitLab % git branch -a
feature
* main
remotes/origin/HEAD -> origin/main
remotes/origin/main
```

二、实验步骤

1. 配置 SSH 并获取仓库

用 `ssh-keygen` 生成 Ed25519 密钥对，并把公钥添加到 GitHub 账号。先验证连接是否正常：

```
Admin@MacBook-Pro GitLab % ssh -T git@github.com
Hi YuanShiWuYu123! You've successfully authenticated, but GitHub does not provide shell access.
```

然后按照指导文件创建个人仓库并克隆到本地。

2. 修改 `main.c` 并提交

把 main.c 中 TODO 处的输出改为 Hello, C++!, 然后提交, 对应提交 2b65734。

```
C main.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("Hello, C++!\n");
7  }
```

3. 新建 feature 分支并提交

```
Admin@MacBook-Pro GitLab % git switch -c feature
Switched to a new branch 'feature'
```

在 feature 分支上修改 main.c, 增加一行输出后提交, 对应提交 f9ac642。

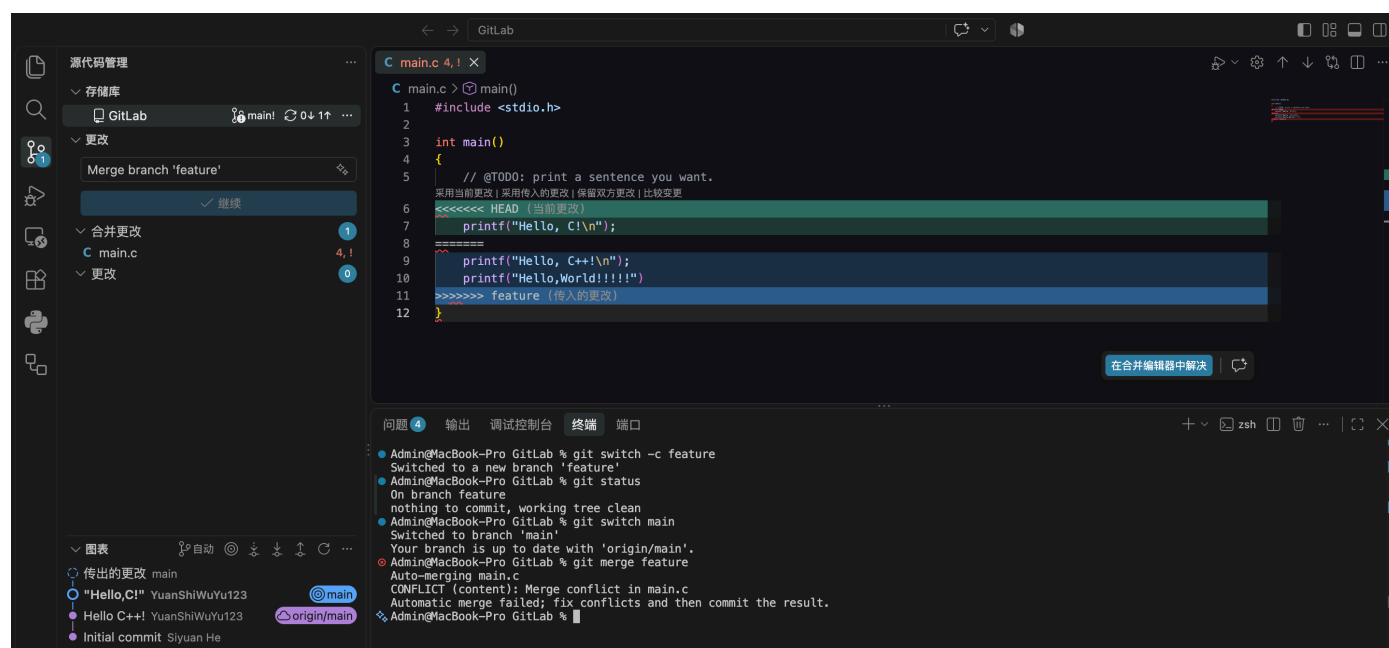
```
C main.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("Hello, C++!\n");
7      printf("Hello,World!!!!");
8  }
```

4. 在 main 分支上提交

切回 main 分支, 把同一行输出改为 Hello, C!, 然后提交, 对应提交 2afd3f8。两个分支修改的是同一行, 为后面产生冲突做准备。

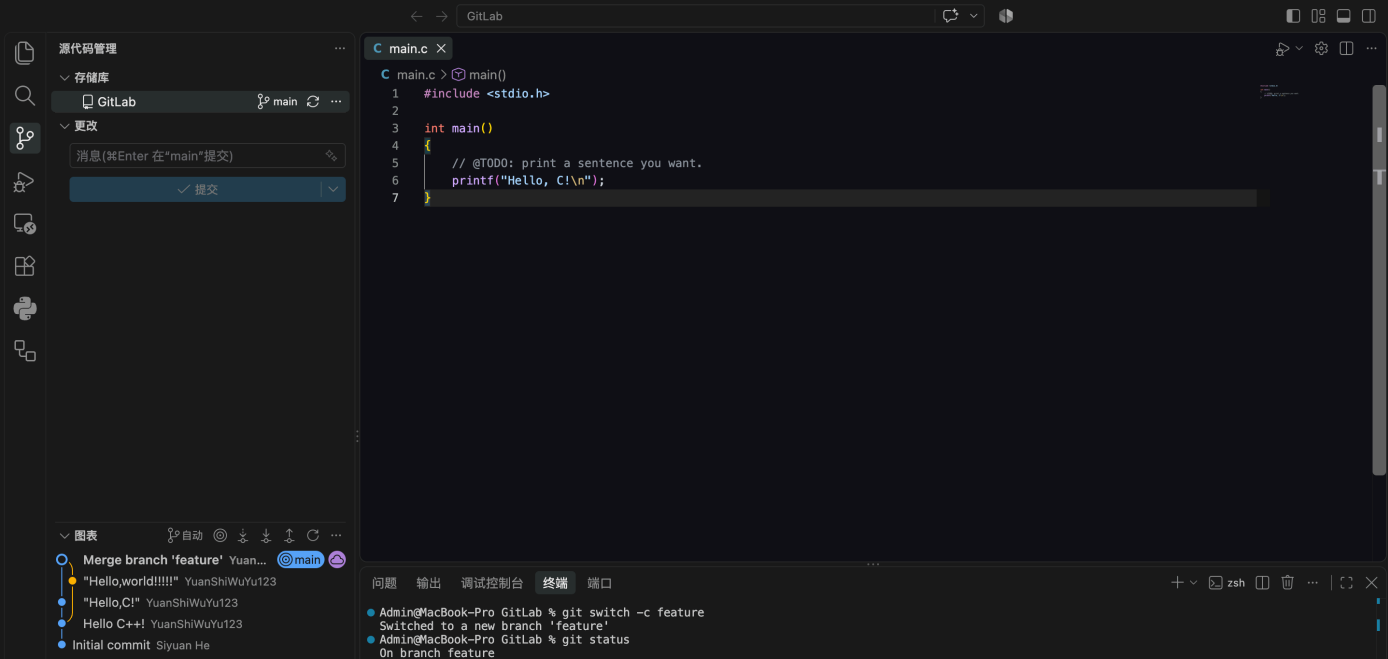
```
C main.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("Hello, C!\n");
7  }
```

5. 合并并处理冲突



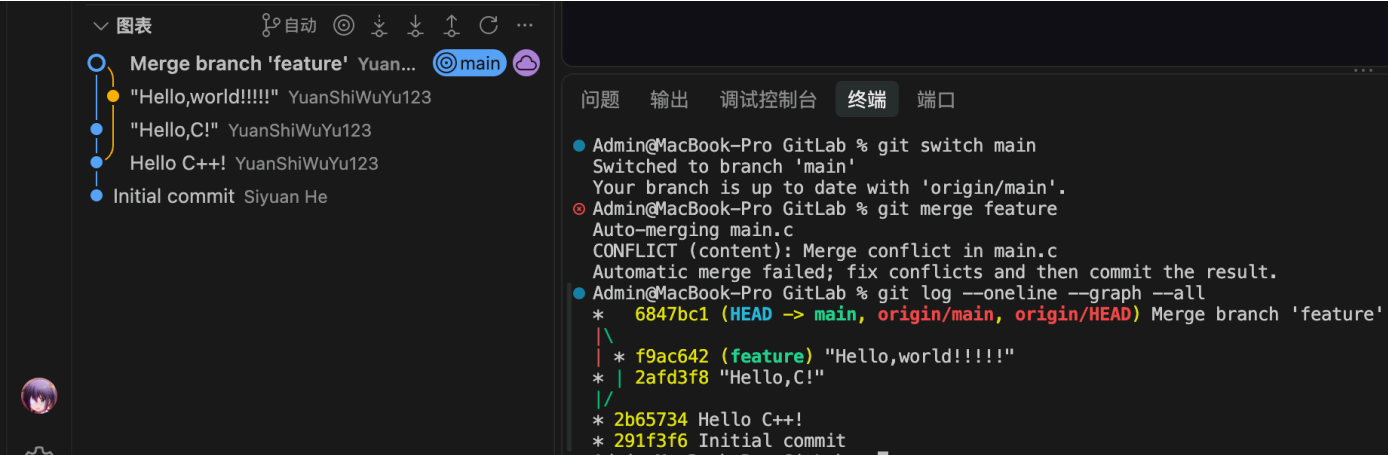
Git 无法自动判断该保留哪一边的修改，在 `main.c` 中插入了冲突标记，其中 HEAD 到分隔线之间是当前分支 `main` 的内容，分隔线到 `feature` 之间是被合并分支的内容。

对比两边的修改后，决定保留 main 分支的版本。在 VS Code 的合并编辑器中选择采用当前更改，删除全部冲突标记，然后提交：



6. 验证并推送

最终的提交历史如下，可以看到两个分支先分叉后合并：



三、阅读概括与思考

1. Commit message 和 Change log 编写指南

文章介绍了提交说明的书写规范，主要讲 Angular 团队使用的格式。一条提交说明分为 Header、Body 和 Footer 三部分，其中 Header 是必需的，格式为 type(scope): subject。type 表示本次提交的类别，常用的有 feat 新功能、fix 修复缺陷、docs 修改文档、refactor 重构等。subject 是一句话概述，使用动词原形开头。Body 说明修改的原因和前后差异，Footer 用于记录不兼容变动或者关闭的 issue。规范化的好处在于，提交历史可以按类型筛选和检索，也可以用工具自动生成版本更新日志。

2. 语义化版本

规范定义了版本号的格式，即主版本号.次版本号.修订号。做出不兼容的 API 修改时主版本号加一，新增向下兼容的功能时次版本号加一，修复缺陷时修订号加一，并且右边的位要归零。0.y.z 表示初始开发阶段，接口随时可能变动，1.0.0 意味着公开 API 正式确定。此外还规定了先行版本号和版本编译信息的写法，以及版本之间的比较规则。这套规则解决的是依赖管理的混乱问题。版本号本身携带了兼容性信息，使用者看到版本号的变化，就能判断这次升级有没有风险。

3. 为什么要学习 Git

最初我以为版本控制就是备份，用来防止代码写坏了找不回来。读完这两篇文章后，我的看法有了变化。Git 记录的不只是文件的状态，还有每一次修改的意图。规范的提交说明让历史可以被检索和理解，提交类型又可以映射到语义化版本号，进而自动生成更新日志。也就是说，从每一次 commit 到最终发布的版本号，是一条完整的链条，而 Git 是这条链条的起点。

对协作来说，Git 的意义更明显。分支让每个人可以在互不干扰的情况下开发，合并时的冲突不是麻烦，而是 Git 在提醒我们两个人改了同一处代码，需要人工确认。这次实验中解决冲突的过程让我体会到，Git 并不试图替人做决定，它只负责把差异准确地呈现出来。